

Traversing a tree:

Traversing a tree is to pass through the tree, visiting each node and print the contents of each node.

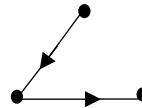
The order in which the nodes are visited in a traversal is clearly from first to last. However, there is no such natural ordering for the nodes of a tree. Thus, different orderings are used for traversal in different cases.

There are three tree traversing methods:

1. Preorder Traversal
2. Inorder Traversal
3. Postorder Traversal

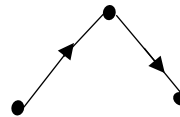
To traverse a nonempty binary tree in preorder, we perform the following three operations.

1. Visit the root.
2. Traverse the left subtree in preorder.
3. Traverse the right subtree in preorder.



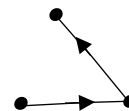
To traverse a nonempty binary tree in inorder, we perform the following three operations.

1. Traverse the left subtree in inorder.
2. Visit the root.
3. Traverse the right subtree in inorder.

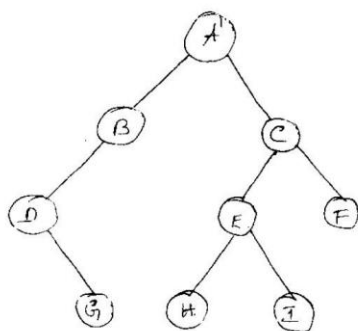


To traverse a nonempty binary tree in postorder, we perform the following three operations.

1. Traverse the left subtree in postorder.
2. Traverse the right subtree in postorder.
3. Visit the root.



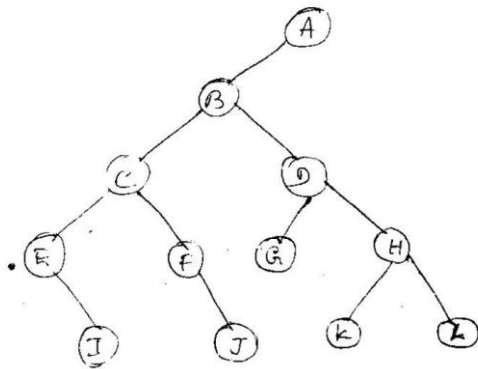
Example:



Preorder Traversal: A B D G C E H I F

Inorder Traversal: D G B A H E I C F

Postorder Traversal: G D B H I E F C A



Preorder Traversal: A B C E I F J D G H K L

Inorder Traversal: E I C F J B G D K H L A

Postorder Traversal: I E J F C G K L H D B A

Binary Tree Traversals in C:

```
void inorder(nodeptr p)
{
    if(p!=NULL)
    {
        inorder(p->left);
        printf("%d\t",p->info);
        inorder(p->right);
    }
}

void preorder(nodeptr p)
{
    if(p!=NULL)
    {
        printf("%d\t",p->info);
        inorder(p->left);
        inorder(p->right);
    }
}

void postorder(nodeptr p)
{
    if(p!=NULL)
    {
```

```
    inorder(p->left);
    inorder(p->right);
    printf("%d\t",p->info);
}
}
```